

Practices

Practice 1 - Data Provider

Prerequisites

You have checked out “lessons/04-asynchronous-processes” or successfully finished chapter 1 of the training practices..

Motivation

Understand Lazy Loading

Time

90 Minuten

Task 1

Create a class `FilterableDataProvider<T>` (we will cover filtering later) in the service module and add 2 methods:

- `Stream<T> fetch(int offset, int limit)`
- `long count()`

create a method in class `TaskService`, which returns a `FilterableDataProvider` for `Task` objects. Create more repository methods if necessary.

Now forward the data provider instead of the `Task` to the UI and transfer your new `DataProvider` into a Vaadin `DataProvider` and set it using `setItems()` to the Grid.

Task 2

Enhance the `FilterableDataProvider` by a sorting function. For this create an own class `SortOrder`, which will help to transfer sorting information from Vaadin to Spring. This class should contain 2 properties:

- `property` - String
- `direction` - Enum (ASC, DESC)

Now add a list of `SortOrder` elements to the method `fetch` of the `FilterableDataProvider`. Now we can convert the query sort information accessible via `Query#getSortOrders()` to our new list of `SortOrder` and use it in our newly enhanced `fetch` method. To make sorting work, we need to process this information now in the persistence module and convert it into something which is understood by our query method. Currently the Grid does not show sort handles yet. We can achieve that by adding

`.setSortProperty(String)` to our column creation. The property we hand over should reflect the class property of `TaskJpa`. The Vaadin Grid will add those automatically to the query object.

Task 3

Enhance the `FilterableDataProvider` by adding a filtering option. For this create a class `TaskFilter` with 3 properties:

- `search` - `String`
- `min` - `LocalDate`
- `max` - `LocalDate`

add the object `TaskFilter` as argument to the methods `count` and `fetch` of the `FilterableProvider` and use these to refine the query in the persistence module. To be able to use this new filtering possibility, we need to add some filter elements to the UI. as a simple start, we can append a second header to our Grid where we add a `TextField` in column "Description" and two `DatePicker` for the column "Due Date". Add a `ValueChangeListener` to every filter component to trigger the filter change. within this Listener you create a filter bean using your 3 components and add it to the `FilterableDataProvider`. After that you need to call `taskGrid.getDataProvider().refreshAll()` to reset the Grid data provider.

Practice 2 - Complex Components & Binder

Prerequisites

Practice 1 is completed or branch "lessons/05-data-providers" is checked out.

Motivation

Understand complex components and binders.

Short Cut

Branch "lessons/05-binder-prepared" is checked out. Continue with Task 3.

Time

120 Minuten

Task 1

Create a bean and a repository for the following structure:

- Person
 - PhoneNumber
 - Country Code
 - Area Code
 - Number
 - Address
 - Address 1
 - Address 2
 - Zip Code
 - City
 - Country

Please pay attention that you don't pull persistence classes to upper layers. To bypass this you can create a method `create()` in a repository and hand it up or you add methods `add(...)` which take in the classes properties to construct the methods within the parent object. I.E.: `Person#addAddress(String address1, ...)`.

Task 2

Create a view having a Grid to show the contacts on the left and an editor for a contact on the right. On selection of a contact in the grid (single select) it should be taken over to the editor side where you can edit it. A new Button in the top will allow to set an empty Person object to the editor, so it can be filled with data and saved.

Task 3

Implement the editor. It shall concise of 3 areas:

1. Changing the name of the Person
2. Changing the PhoneNumber
 - Complex component with `HasValue<E, PhoneNumber>`
3. Changing the Address
 - Complex component with `HasValue<E, Address>`

in the bottom there are 2 Buttons. One to revert and one to save.